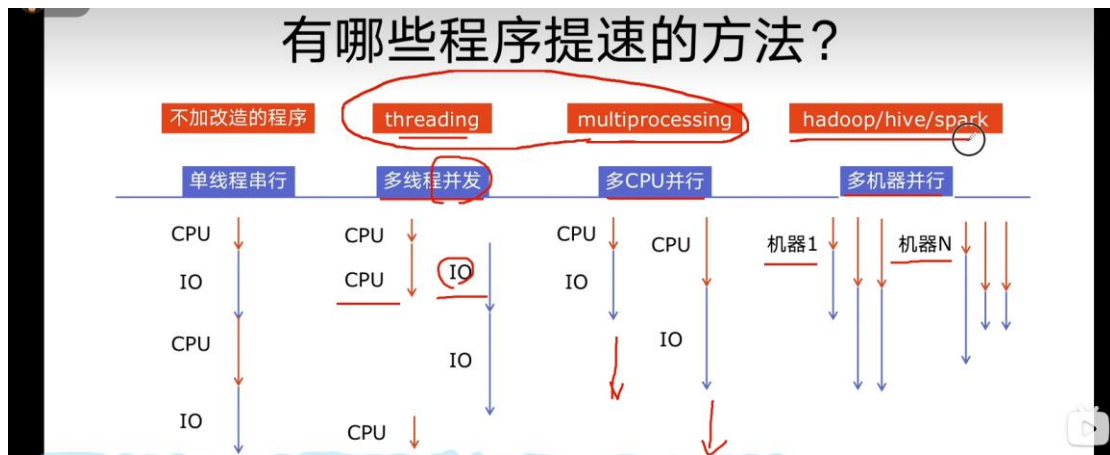


1.0 概述



让 IO 不再占用 cpu 时间;

Python对并发编程的支持

- 多线程: threading, 利用CPU和IO可以同时执行的原理, 让CPU不会干巴巴等待IO完成
- 多进程: multiprocessing, 利用多核CPU的能力, 真正的并行执行任务
- 异步IO: asyncio, 在单线程利用CPU和IO同时执行的原理, 实现函数异步执行
- 使用Lock对资源加锁, 防止冲突访问
- 使用Queue实现不同线程/进程之间的数据通信, 实现生产者-消费者模式
- 使用线程池Pool/进程池Pool, 简化线程/进程的任务提交、等待结束、获取结果
- 使用subprocess启动外部程序的进程, 并进行输入输出交互

多线程是在线程 1IO 时,CPU 执行其他线程,为线程级并发;而异步 IO 是让单线程在 IO 时,CPU 继续执行不需 IO 的函数,为函数级并发;

2.0 前置知识

- 1、什么是CPU密集型计算、IO密集型计算？
- 2、多线程、多进程、多协程的对比
- 3、怎样根据任务选择对应技术？

2.1.1

1、什么是CPU密集型计算、IO密集型计算？

CPU密集型 (CPU-bound) :

CPU密集型也叫计算密集型,是指I/O在很短的时间就可以完成,CPU需要大量的计算和处理,特点是CPU占用率相当高

例如: 压缩解压缩、加密解密、正则表达式搜索

IO密集型 (I/O bound) :

IO密集型指的是系统运作大部分的状况是CPU在等I/O (硬盘/内存) 的读/写操作,CPU占用率仍然较低。

例如: 文件处理程序、网络爬虫程序、读写数据库程序

若程序大量依赖内存,网络,数据等为 IO 密集型;否则为 CPU 密集型;

2.1.2

2、多线程、多进程、多协程的对比

一个进程中
可以启动N个线程

一个线程中
可以启动N个协程

多进程 Process (multiprocessing)

- 优点: 可以利用多核CPU并行运算
- 缺点: 占用资源最多、可启动数目比线程少
- 适用于: CPU密集型计算

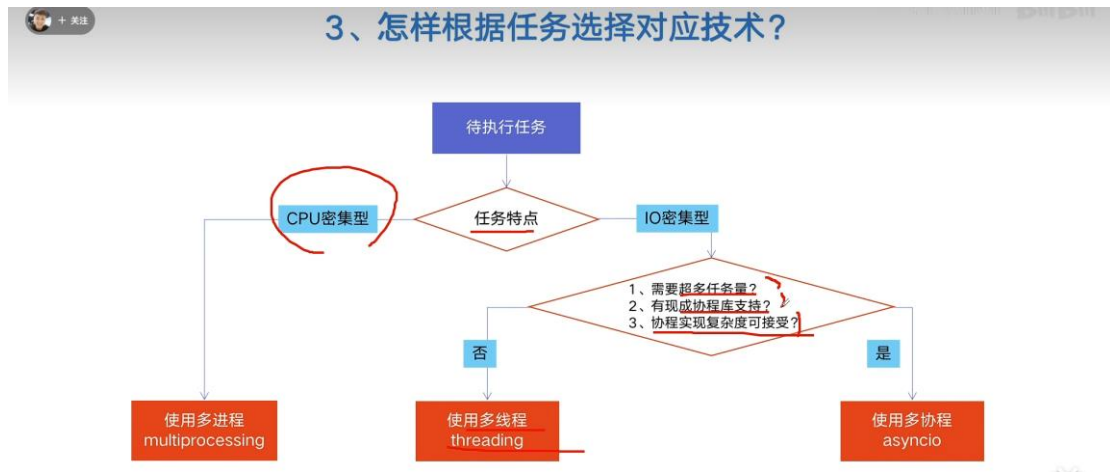
多线程 Thread (threading)

- 优点: 相比进程, 更轻量级、占用资源少
- 缺点:
 - 相比进程: 多线程只能并发执行, 不能利用多CPU (GIL)
 - 相比协程: 启动数目有限制, 占用内存资源, 有线程切换开销
- 适用于: IO密集型计算、同时运行的任务数目要求不多

多协程 Coroutine (asyncio)

- 优点: 内存开销最少、启动协程数量最多
- 缺点: 支持的库有限制 (aiohttp vs requests)、代码实现复杂
- 适用于: IO密集型计算、需要超多任务运行、但有现成库支持的场景

2.1.3



2.2 GIL

- 1、Python速度慢的两大原因
- 2、GIL是什么?
- 3、为什么有GIL这个东西?
- 4、怎样规避GIL带来的限制?

1、Python速度慢的两大原因

相比C/C++/JAVA, Python确实慢, 在一些特殊场景下, Python比C++慢100~200倍

由于速度慢的原因, 很多公司的基础架构代码依然用C/C++开发
比如各大公司阿里/腾讯/快手的推荐引擎、搜索引擎、存储引擎等底层对性能要求高的模块

Python 速度慢的原因1

动态类型语言
边解释边执行

Python 速度慢的原因2

GIL
无法利用多核CPU并发执行

2、GIL 是什么？

全局解释器锁（英语：Global Interpreter Lock，缩写GIL）

是计算机程序设计语言解释器用于同步线程的一种机制，它使得任何时刻仅有一个线程在执行。

即便在多核心处理器上，使用 GIL 的解释器也只允许同一时间执行一个线程。

由于GIL的存在
即使电脑有多核CPU
单个时刻也只能使用1个
相比并发加速的C++/JAVA所以慢

- When a thread is running, it holds the GIL
- GIL released on I/O (read,write,send,recv,etc.)

多核 CPU,c/c++可以并行运行多个线程

3、为什么有GIL这个东西？

简而言之：Python设计初期，为了规避并发问题引入了GIL，现在想去却去不掉了！

为了解决多线程之间数据完整性和状态同步问题

原因详解 Python中对象的管理，是使用引用计数器进行的，引用数为0则释放对象

开始：线程A和线程B都引用了对象obj，obj.ref_num = 2，线程A和B都想撤销对obj的引用

线程A

线程B

obj.ref_num -- 变成1 此时发生多线程调度切换

obj.ref_num --, 变成0

if obj.ref_num == 0: free obj

此时发生多线程调度切换

if obj.ref_num == 0: free obj 错误: obj已经不存在了 这两行代码可能破坏内存

GIL确实有好处:
简化了Python对共享资源的管理;

规避了共享资源的同步问题;

4、怎样规避GIL带来的限制？

- 多线程 threading 机制依然是有用的，用于IO密集型计算

因为在 I/O (read,write,send,recv,etc.)期间，线程会释放GIL，实现CPU和IO的并行

因此多线程用于IO密集型计算依然可以大幅提升速度

但是多线程用于CPU密集型计算时，只会更加拖慢速度

- 使用multiprocessing 的多进程机制实现并行计算、利用多核CPU优势

为了应对GIL的问题，Python提供了multiprocessing

拖慢速度:线程切换也存在开销;

3.0 单线程多线程实战:爬虫时间对比

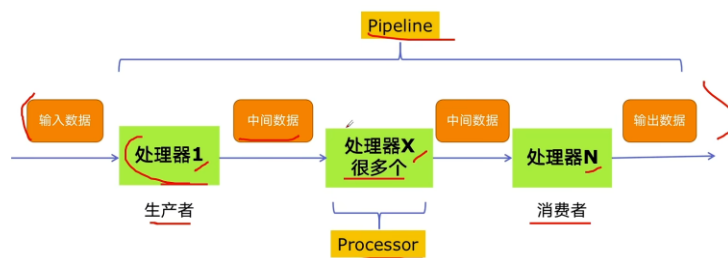
见项目 py_multiprocess

4.0 多线程生产者消费者

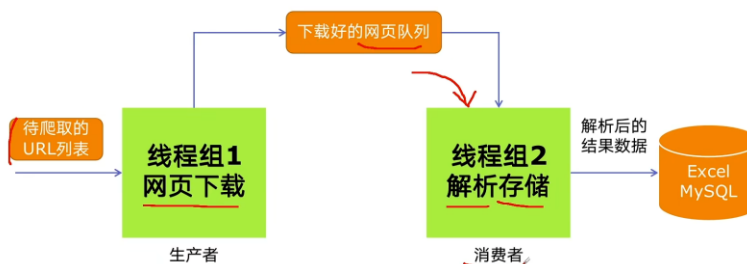
- 1、多组件的Pipeline技术架构
- 2、生产者消费者爬虫的架构
- 3、多线程数据通信的queue.Queue
- 4、代码编写实现生产者消费者爬虫

1、多组件的Pipeline技术架构

复杂的事情一般都不会一下子做完，而是会分很多中间步骤一步步完成



2、生产者消费者爬虫的架构



3、多线程数据通信的queue.Queue

queue.Queue可以用于多线程之间的、线程安全的数据通信

```
1、导入类库
import queue

2、创建Queue
q = queue.Queue()

3、添加元素
q.put(item)

4、获取元素
item = q.get()
```

5、查询状态

```
# 查看元素的多少
q.qsize()
# 判断是否为空
q.empty()
# 判断是否已满
q.full()
```

queue 特点: 当 q 满了,则 Put()阻塞卡住;当 q 空,则 get()卡住;

代码见项目 py_multiProcess/provider_consumer_test

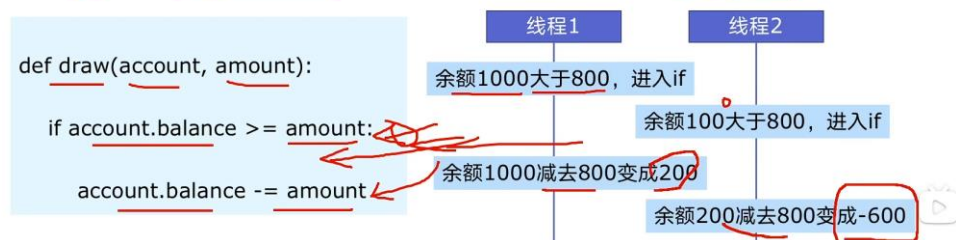
5.0

- 1、线程安全概念介绍
- 2、Lock 用于解决线程安全问题
- 3、实例代码演示问题 以及 解决方案

1、线程安全概念介绍

线程安全指某个函数、函数库在多线程环境中被调用时，能够正确地处理多个线程之间的共享变量，使程序功能正确完成。

由于线程的执行随时会发生切换，就造成了不可预料的结果，出现线程不安全



2、Lock 用于解决线程安全问题

用法1: try-finally模式

```
import threading  
  
lock = threading.Lock()  
  
lock.acquire()  
try:  
    # do something  
finally:  
    lock.release()
```

用法2: with 模式

```
import threading  
  
lock = threading.Lock()  
  
with lock:  
    # do something
```

通过上锁解决时间错误;

代码见 `py_multiProcess/lock_test/lock.py`

6.0

- 1、线程池的原理
- 2、使用线程池的好处
- 3、ThreadPoolExecutor的使用语法
- 4、使用线程池改造爬虫程序

1、线程池的原理



线程池存放提前建好的线程,任务队列中有任务,执行完后不销毁,而是重新进入线程池;实现了规避线程的建毁开销;

2、使用线程池的好处

- 1、提升性能：因为减去了大量新建、终止线程的开销，重用了线程资源；
- 2、适用场景：适合处理突发性大量请求或需要大量线程完成任务、但实际任务处理时间较短
- 3、防御功能：能有效避免系统因为创建线程过多，而导致系统负荷过大相应变慢等问题
- 4、代码优势：使用线程池的语法比自己新建线程执行线程更加简洁

3、ThreadPoolExecutor的使用语法

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
with ThreadPoolExecutor() as pool:
```

```
    results = pool.map(crawl, urls)
```

```
    for result in results:  
        print(result)
```

用法1: map函数, 很简单
注意map的结果和入参是顺序对应的

```
with ThreadPoolExecutor() as pool:
```

```
    futures = [ pool.submit(crawl, url)  
                for url in urls ]
```

```
    for future in futures:  
        print(future.result())  
    for future in as_completed(futures):  
        print(future.result())
```

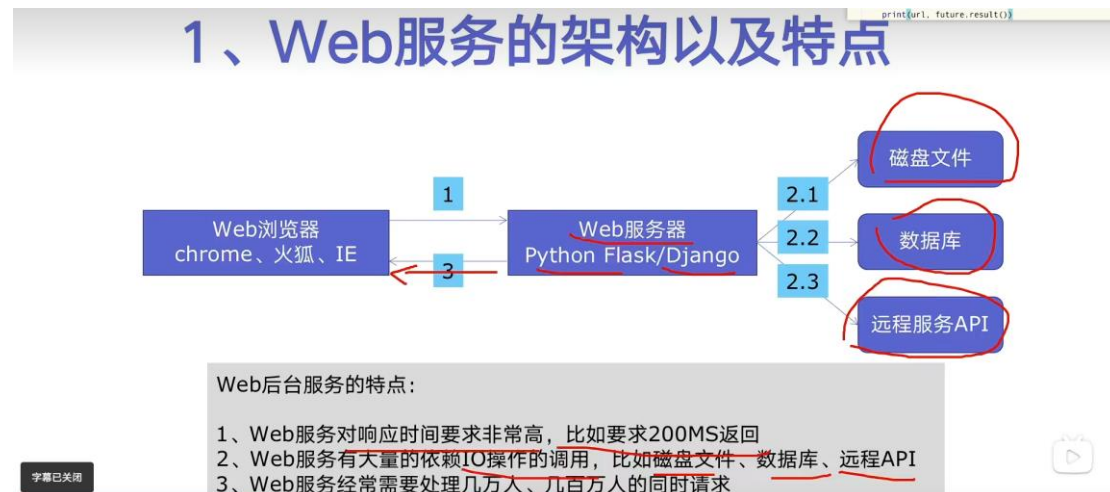
用法2: future模式, 更强大
注意如果用as_completed顺序是不定的

map 方法中的 urls 为参数(单个参数/元组)组成的列表;results 中结果和参数顺序对应;
submit 方法第一种遍历要等所有结果出来后打印;而第二种方法,先出结果先打印;
代码见:py_multiProcess/thread_pool_test/pool_test.py

总结:向数据库,外存,api 中获取数据的函数 都可以交给线程池优化;

7.0

- 1、Web服务的架构以及特点
- 2、使用线程池ThreadPoolExecutor加速
- 3、代码用Flask实现Web服务并实现加速



2、使用线程池ThreadPoolExecutor加速

使用线程池ThreadPoolExecutor的好处：

- 1、方便的将磁盘文件、数据库、远程API的IO调用并发执行
- 2、线程池的线程数目不会无限创建（导致系统挂掉），具有防御功能

代码见:py_multiProcess/web_pool_test/flask_pool.py

8.0

- 1、有了多线程threading，为什么还要用多进程multiprocessing
- 2、多进程multiprocessing知识梳理
- 3、代码实战：单线程、多线程、多进程对比CPU密集计算速度

1、有了多线程threading，为什么还要用多进程multiprocessing